

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters: A Machine Learning Approach to Flood Prediction is an intelligent disaster management application that leverages Machine Learning to predict the likelihood of flood occurrences using historical weather and rainfall data. The system aims to provide timely and accurate flood risk predictions, enabling authorities and communities to take preventive measures and reduce the impact of natural disasters.

The application incorporates modules for data preprocessing, visualization, model training, and real-time flood prediction. Multiple classification algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, are evaluated to identify the most accurate model. The best-performing XGBoost model, achieving **96.55% accuracy**, is deployed through a Flask-based web application.

Built using **Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn**, the system provides a simple and user-friendly interface where users can enter rainfall and meteorological parameters to receive instant flood predictions. The application is designed for deployment on **IBM Cloud**, making it scalable and accessible to disaster management authorities from any location.

Scenarios

Scenario 1: Early Flood Warning and Evacuation Planning

A meteorologist enters current annual rainfall, seasonal rainfall, and cloud visibility data for a flood-prone district through the web application. The XGBoost model analyzes the weather conditions and predicts a high probability of flooding, allowing authorities to issue early warnings and evacuation notices before the disaster occurs.

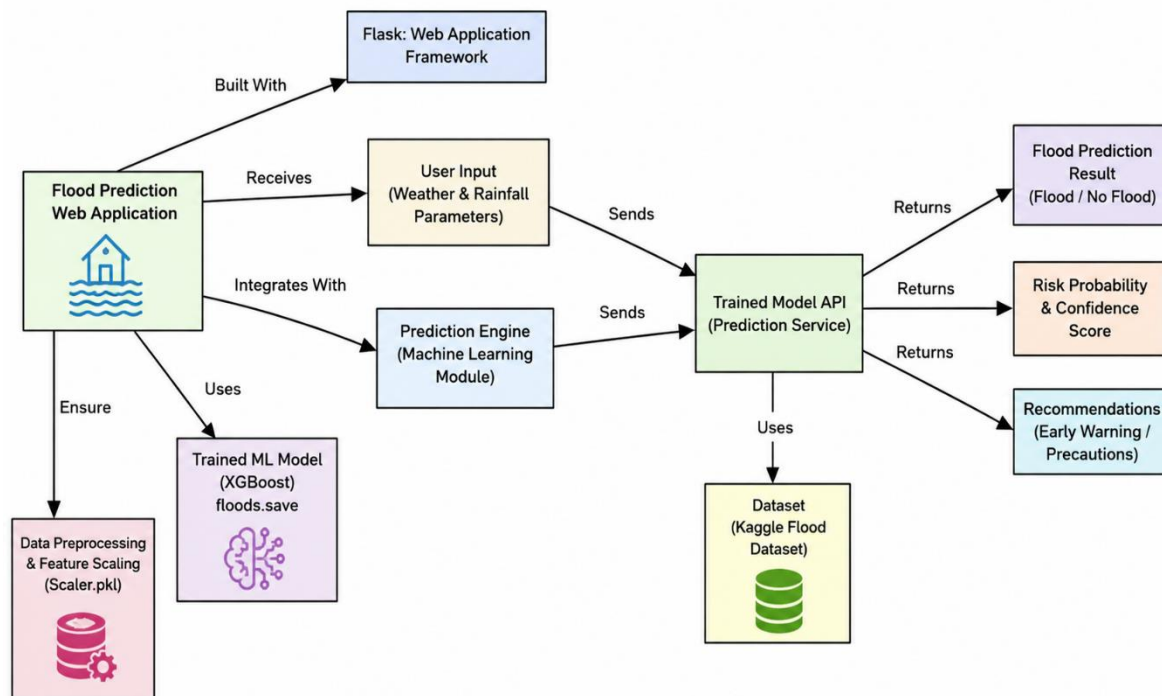
Scenario 2: Disaster Response and Resource Allocation

During the monsoon season, a disaster management officer monitors weather conditions across multiple regions by entering the latest meteorological data into the system. The application instantly predicts flood risk levels, enabling emergency response teams to prioritize rescue operations, allocate resources efficiently, and prepare relief measures for high-risk areas.

Scenario 3: Flood Prediction Model Validation

A government analyst evaluates the trained machine learning model using historical flood datasets collected from Kaggle. After testing various classification algorithms, the XGBoost model achieves an accuracy of **96.55%**, confirming its reliability and effectiveness for real-world flood prediction and disaster preparedness.

Technical Architecture:



Description: **Rising Waters** uses a modular three-tier architecture to provide accurate flood prediction using Machine Learning. The frontend, built with **HTML, CSS, and Flask**, collects rainfall and weather-related inputs from users. The **Flask backend** validates the input data and processes prediction requests. The trained **XGBoost model** analyzes the data to predict flood risk with high accuracy. Prediction results are displayed instantly through the web application. The system is designed for scalable deployment on **IBM Cloud** to support disaster preparedness and early flood warning.

Pre-requisites:

- **Flask Framework Knowledge:** <https://flask.palletsprojects.com/>
- **Python Programming Proficiency:** <https://docs.python.org/3/>
- **Machine Learning with Scikit-learn:** <https://scikit-learn.org/stable/>
- **NumPy Library:** <https://numpy.org/doc/>
- **Pandas Library:** <https://pandas.pydata.org/docs/>
- **Matplotlib Library:** <https://matplotlib.org/stable/users/index.html>
- **Seaborn Library:** <https://seaborn.pydata.org/>
- **HTML, CSS, and JavaScript Skills:** <https://www.w3schools.com/>
- **Version Control with Git:** <https://git-scm.com/doc>
- **Flask Development Environment Setup:** <https://flask.palletsprojects.com/en/stable/installation/>
- **IBM Cloud Documentation (Deployment):** <https://cloud.ibm.com/docs>
- **Kaggle Dataset Platform:** <https://www.kaggle.com/datasets>

Project Workflow:

Milestone 1: Data Collection, Model Selection and Architecture

- **Activity 1.1:** Set up the development environment using Python, Anaconda/Jupyter Notebook, and install the required libraries (NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, Flask, Joblib).
- **Activity 1.2:** Collect the flood prediction dataset from Kaggle and perform data preprocessing, including handling missing values and feature scaling.
- **Activity 1.3:** Train and evaluate multiple Machine Learning models (Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost) and select the best-performing model.
- **Activity 1.4:** Design the system architecture, defining interactions between the user interface, Flask backend, Machine Learning model, and prediction module.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the data preprocessing, visualization, and model prediction modules.
- **Activity 2.2:** Implement the Flask backend to process user inputs, load the trained XGBoost model, and generate real-time flood predictions.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in **app.py**, configure Flask routes, integrate the trained model, and display flood prediction results through the web application.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop a responsive user interface using **HTML, CSS, and JavaScript** for entering weather and rainfall parameters.
- **Activity 4.2:** Create dynamic Flask templates using **render_template()** to display the Home page, Prediction page, Flood Chance Result page, and No Flood Chance Result page.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for deployment by configuring the Flask environment and ensuring all dependencies are installed.
- **Activity 5.2:** Deploy the application on **IBM Cloud** (or run locally using the Flask development server) and verify that the prediction system is accessible and functioning correctly.

Milestone 6: Conclusion

- **Activity 6.1:** Perform system testing and validate the prediction accuracy of the XGBoost model.
- **Activity 6.2:** Finalize the project documentation, demonstrate the application, and submit the completed **Rising Waters: A Machine Learning Approach to Flood Prediction** project.

Milestone 1: Model Selection and Architecture

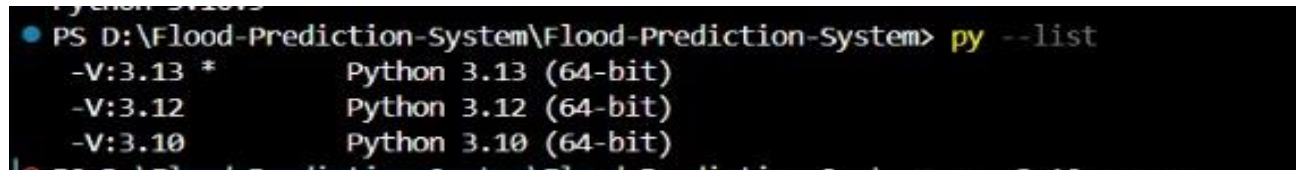
In this milestone, we focus on selecting the appropriate Machine Learning model for flood prediction. This involves setting up the development environment, collecting and analyzing the flood prediction dataset, preprocessing the data, comparing multiple classification algorithms, and selecting the best-performing model. The overall system architecture is also designed to define the interaction between the Flask web application, Machine Learning model, and users.

Activity 1.1: Set Up the Development Environment

Before developing the Flood Prediction System, install the required software and Python libraries.

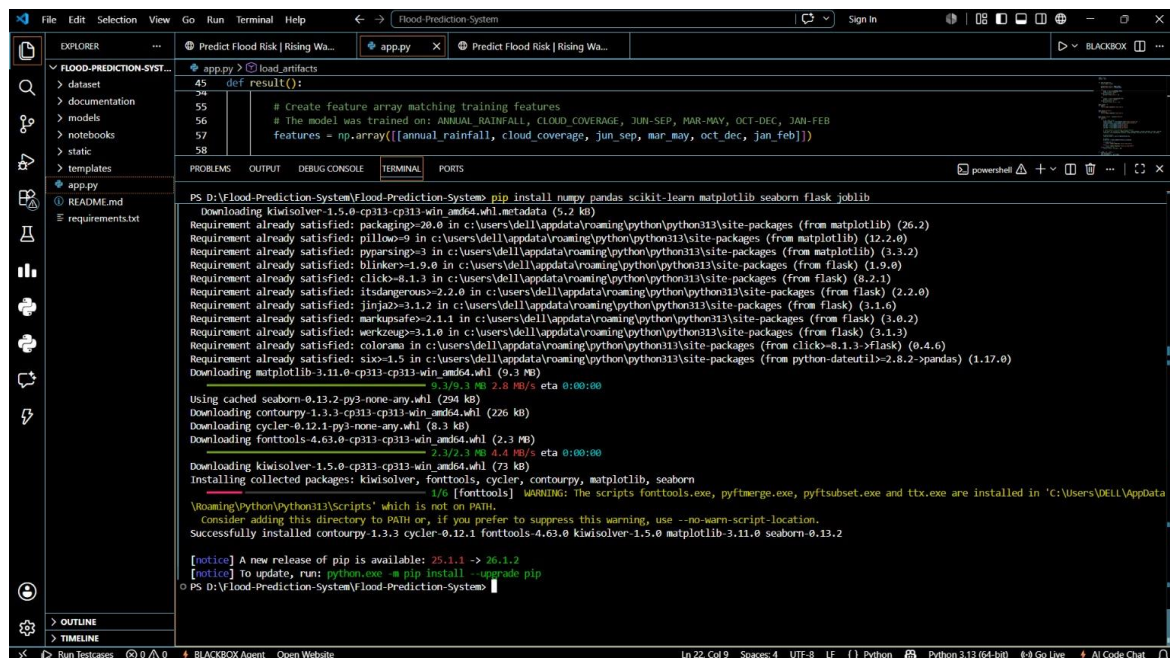
Step 1 - Install Python and Anaconda

Download and install Python 3.8 or above and Anaconda Navigator. These provide the environment required for data analysis, machine learning, and Flask application development.



```
PS D:\Flood-Prediction-System\Flood-Prediction-System> py --list
-V:3.13 * Python 3.13 (64-bit)
-V:3.12 Python 3.12 (64-bit)
-V:3.10 Python 3.10 (64-bit)
```

Step 2 - Install Required Libraries Install all the required Python packages using pip or Anaconda Prompt.



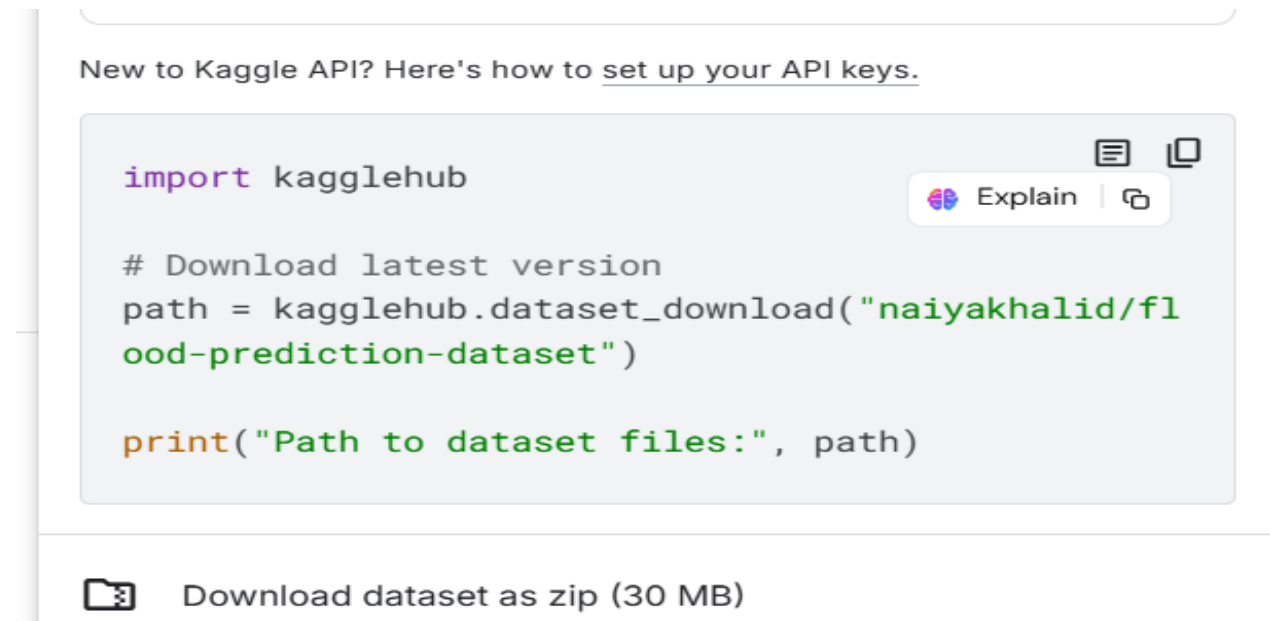
```
PS D:\Flood-Prediction-System\Flood-Prediction-System> pip install numpy pandas scikit-learn matplotlib seaborn flask joblib
Downloading kiwisolver-1.5.0-cp313-cp313-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: packaging>20.0 in c:\users\dell\appdata\roaming\python\python313\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>9 in c:\users\dell\appdata\roaming\python\python313\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>3 in c:\users\dell\appdata\roaming\python\python313\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: blinker>1.9.0 in c:\users\dell\appdata\roaming\python\python313\site-packages (from flask) (1.9.0)
Requirement already satisfied: click>8.1.3 in c:\users\dell\appdata\roaming\python\python313\site-packages (from flask) (8.2.1)
Requirement already satisfied: itsdangerous>2.2.0 in c:\users\dell\appdata\roaming\python\python313\site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>3.1.2 in c:\users\dell\appdata\roaming\python\python313\site-packages (from flask) (3.1.0)
Requirement already satisfied: markupsafe>2.1.1 in c:\users\dell\appdata\roaming\python\python313\site-packages (from flask) (3.0.2)
Requirement already satisfied: werkzeug>3.1.0 in c:\users\dell\appdata\roaming\python\python313\site-packages (from flask) (3.1.3)
Requirement already satisfied: colorama in c:\users\dell\appdata\roaming\python\python313\site-packages (from click>8.1.3->flask) (0.4.6)
Requirement already satisfied: six>1.5 in c:\users\dell\appdata\roaming\python\python313\site-packages (from python-dateutil>2.8.2->pandas) (1.17.0)
Downloading matplotlib-3.11.0-cp313-cp313-win_amd64.whl (9.3 MB)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Downloading contourpy-1.3.3-cp313-cp313-win_amd64.whl (226 kB)
Downloading cycycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.63.0-cp313-cp313-win_amd64.whl (2.3 MB)
Downloading kiwisolver-1.5.0-cp313-cp313-win_amd64.whl (73 kB)
Installing collected packages: kiwisolver, fonttools, cycycler, contourpy, matplotlib, seaborn
Successfully installed contourpy-1.3.3 cycycler-0.12.1 fonttools-4.63.0 kiwisolver-1.5.0 matplotlib-3.11.0 seaborn-0.13.2
```

pip install numpy pandas scikit-learn matplotlib seaborn flask joblib

These libraries are used for data preprocessing, visualization, model training, and web application development.

Step 3 -Download the Flood Prediction Dataset

Download the **Flood Prediction Dataset** from Kaggle and place it in the project directory. The dataset contains rainfall and meteorological parameters used for training the Machine Learning model.



Step 4 -Train and Select the Best Machine Learning Model

Train multiple classification algorithms, including **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **XGBoost**. Compare their performance using accuracy score, confusion matrix, and classification report.

The **XGBoost** model achieved the highest accuracy (**96.55%**) and was selected for deployment.

Step 5 - Save the Trained Model

Save the trained XGBoost model using **Joblib/Pickle** for deployment in the Flask. The saved model is loaded by the Flask application to perform real-time flood predictions.

```
import joblib
joblib.dump(model, "floods.save")import joblib
```

Step 6 - Design the System Architecture

Design the overall system architecture illustrating the interaction between the User Interface, Flask Backend, Machine Learning Model, and Flood Prediction Result. This architecture forms the foundation for the application's implementation and deployment.

Activity 1.2: Research and Select the Appropriate Machine Learning Model

The following activities were carried out to identify the most suitable Machine Learning model for flood prediction:

- **Understand the Project Requirements:** Analyze the project's objective of predicting flood risk accurately using historical rainfall and meteorological data to support early disaster management.
- **Explore Machine Learning Algorithms:** Study various classification algorithms, including **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **XGBoost**, to understand their working principles, strengths, and suitability for flood prediction.
- **Evaluate Model Performance:** Compare the models using performance metrics such as **Accuracy Score**, **Confusion Matrix**, and **Classification Report** to determine the most effective algorithm.
- **Select the Optimal Model:** Based on the evaluation results, **XGBoost** was selected as the final model because it achieved the highest prediction accuracy (**96.55%**) and demonstrated better reliability and generalization than the other models. The trained model was saved as `floods.save` for integration with the Flask web application.
- .

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the **HTML, CSS, and JavaScript frontend**, **Flask backend**, **Machine Learning (XGBoost) model**, and the **prediction result module**.
- **Detail Frontend Functionality:** Define how users interact with the application by entering **rainfall and meteorological parameters** through the prediction form and submitting them to obtain instant flood prediction results.
- **Outline Backend Responsibilities:** Specify how the **Flask backend** receives user input, validates the entered values, preprocesses the data, loads the trained **XGBoost model (floods.save)**, and generates flood predictions.
- **Describe Machine Learning Integration:** Explain how the backend passes the processed input data to the trained **XGBoost model**, receives the prediction output, and displays either **"Flood Chance"** or **"No Flood Chance"** on the result page through the Flask web application.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8 or above is installed along with pip for managing project dependencies.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate project dependencies.

```
D:\projects> python -m venv myenv
```

```
D:\projects> myenv\Scripts\activate
```

```
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Required Python Libraries:** Install all the required libraries for Machine Learning, data analysis, visualization, and Flask.

```
(myenv) D:\projects> pip install flask pandas numpy scikit-learn matplotlib seaborn joblib
```

- **Verify the Installation:**
- Confirm that all required libraries are installed successfully before running the application.

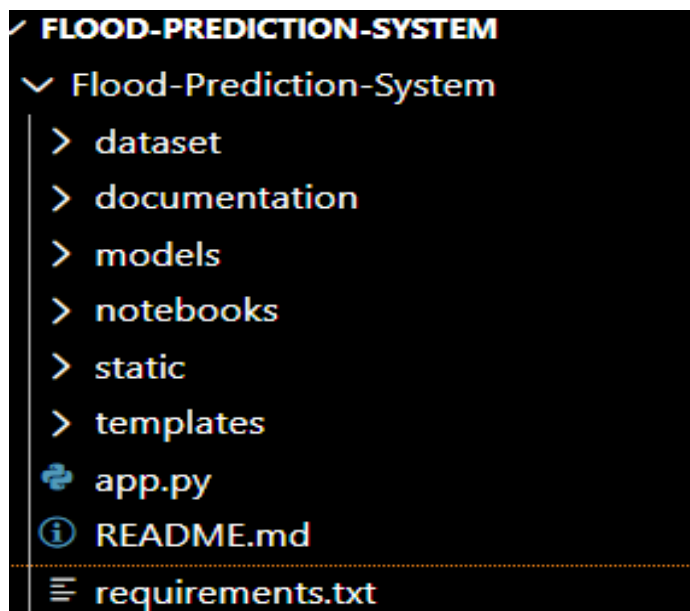
```
(myenv) D:\projects> pip list
```

- **Run the Flask Application:**
- Start the Flask development server to verify that the application is working correctly.

```
(myenv) D:\projects> python app.py
```

- **Set Up the Project Structure:**

Organize the project directory with folders for **templates**, **static**, **datasets**, **trained model (floods.save)**, and the main application files such as **app.py**, **requirements.txt**, and supporting Python scripts.



Milestone 2: Core Functionalities Development

Activity 2.1 Develop Core Flood Prediction Features

Implement the core functionalities of the Flood Prediction System, including data preprocessing, machine learning prediction, and result generation through the Flask web application.

- **Data Preprocessing:** Clean and preprocess the input data by handling missing values, applying feature scaling, and preparing rainfall and meteorological parameters for prediction.
- **Flood Prediction Module:** Integrate the trained **XGBoost** model to analyze user-provided rainfall and weather data and predict the likelihood of flood occurrence with high accuracy.
- **Prediction Result Generation:** Display the prediction outcome as "**Flood Chance**" or "**No Flood Chance**" through a user-friendly web interface, enabling quick interpretation of flood risk.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Configure the required routes in **app.py** for the Home page, Prediction Input page, and Prediction Result page.

Process User Input:

- Create an input form in the HTML template to allow users to enter rainfall and weather-related parameters.
- Use Flask's **request.form** to receive user input from the prediction form.
- Validate that all required input values are entered correctly before processing the prediction request.

Integrate the Machine Learning Model:

- Load the trained **XGBoost model (floods.save)** using **Joblib/Pickle** during application startup.
- Pass the processed user input to the trained model for prediction.
- Receive the prediction result from the model and determine whether there is a **Flood Chance** or **No Flood Chance**.
- Display the prediction result dynamically on the result page using Flask's **render_template()** function.

This implementation ensures smooth interaction between the web interface, Flask backend, and Machine Learning model, providing fast and accurate flood prediction results.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the app.py file, which serves as the backbone of the Rising Waters: A Machine Learning Approach to Flood Prediction application. During this milestone, the Flask application is configured with the required routes, the trained Machine Learning model is integrated, user inputs are validated, and prediction results are generated and displayed through the web interface.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish the required routes for the Flood Prediction application.

Home Route (/)

```
@app.route("/")
def home():
    return render_template("index.html")
```

/Result — POST endpoint that accepts user input, invokes the trained XGBoost Machine Learning model, and returns the flood prediction result

```
app.route('/result', methods=['POST'])
def result():
    try:

        annual_rainfall = float(request.form['annual_rainfall'])
        cloud_coverage = float(request.form['cloud_coverage'])
        jun_sep = float(request.form['jun_sep'])
        mar_may = float(request.form['mar_may'])
        oct_dec = float(request.form['oct_dec'])
        jan_feb = float(request.form['jan_feb'])

        features = np.array([
            annual_rainfall,
```

```
        cloud_coverage,
        jun_sep,
        mar_may,
        oct_dec,
        jan_feb
    ])

    # Scale features
    features_scaled = scaler.transform(features)

    # Predict
    prediction = model.predict(features_scaled)[0]

    if prediction == 1:
        return render_template('result_flood.html')
    else:
        return render_template('result_no_flood.html')

except Exception as e:
    return f'Error: {str(e)}', 400
```

Set Up Route Handlers for Each Feature:

- **For the /result route**, implement logic that reads **annual rainfall, cloud coverage, June–September rainfall, March–May rainfall, October–December rainfall, and January–February rainfall** from the HTML form using **request.form**.
- **Apply input validation** by converting the entered values into floating-point numbers. If invalid or missing values are entered, the application returns an error message (**400 Bad Request**).
- **Process user input** by creating a feature array, scaling the input data using the trained scaler (scaler.pkl), and passing it to the trained **XGBoost** model for prediction.

- **Return the prediction result** by rendering either **result_flood.html** or **result_no_flood.html** based on the model's prediction.
- **Define Model Loading Helper:**
- **load_artifacts()** Function: Loads the trained Machine Learning model (floods.save) and feature scaler (scaler.pkl) during application startup.

```
def load_artifacts():  
    global model, scaler  
    try:  
        model = joblib.load(MODEL_PATH)  
        scaler = joblib.load(SCALER_PATH)  
        print("Model and scaler loaded successfully!")  
    except Exception as e:  
        print(f"Error loading model: {e}")
```

- **Define Input Feature Processing**
- The application accepts the following six input features from the user:

```
annual_rainfall  
cloud_coverage  
jun_sep  
mar_may  
oct_dec
```

- These values are converted into a NumPy array before prediction.
-

```
features = np.array([[annual_rainfall,  
                      cloud_coverage,  
                      jun_sep,  
                      mar_may,  
                      oct_dec,  
                      jan_feb]])
```

- **Integrate the Machine Learning Model:** Scale the input data using the trained **Standard Scaler**.

```
features_scaled = scaler.transform(features)
```

- Pass the scaled data to the trained **XGBoost** model to predict flood occurrence.

```
prediction = model.predict(features_scaled)[0]
```

Display the prediction result based on the model output.

```
if prediction == 1:  
    return render_template('result_flood.html')  
else:  
    return render_template('result_no_flood.html')
```

- **Handle Exceptions:** The application uses a **try-except** block to catch invalid input values or prediction errors and returns an appropriate error message.

```
except Exception as e:  
    return f'Error: {str(e)}', 400
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, responsive, and user-friendly web interface for the Flood Prediction System. The frontend is built using HTML, CSS, and Flask templates to collect weather information from users and display flood prediction results in an easy-to-understand format.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main **Home Page (index.html)** to introduce the Flood Prediction System with navigation to the prediction page.
- Create an **Input Page (input.html)** containing a form for entering flood-related parameters such as Annual Rainfall, Cloud Coverage, Jun–Sep Rainfall, Mar–May Rainfall, Oct–Dec Rainfall, and Jan–Feb Rainfall.
- Design separate result pages (**result_flood.html** and **result_no_flood.html**) to display prediction outcomes.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet to provide a clean, responsive, and user-friendly interface.
- Use responsive layouts so the application works properly on desktop, tablet, and mobile devices.
- Apply appropriate colors, buttons, input fields, and spacing to improve usability.

Create Separate UI Components :

- **Home Page:** Displays project information and prediction navigation.

- **Prediction Form:** Accepts meteorological inputs required by the machine learning model.
- **Flood Result Page:** Displays a message indicating a high probability of flooding.
- **No Flood Result Page:** Displays a message indicating low flood risk.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission :

- Create an HTML form that submits user input to the **/result** route using the POST method.
- Validate user inputs before sending them to the Flask backend.

Display Prediction Results:

- Receive the prediction from the trained machine learning model.
- Redirect users to either **result_flood.html** or **result_no_flood.html** based on the prediction.
- Present the prediction result clearly with user-friendly messages.

Improve User Experience:

- Ensure smooth page navigation between Home, Prediction, and Result pages.
- Display meaningful error messages if invalid values are entered or if prediction fails.
- Maintain a simple and responsive interface for easy accessibility.

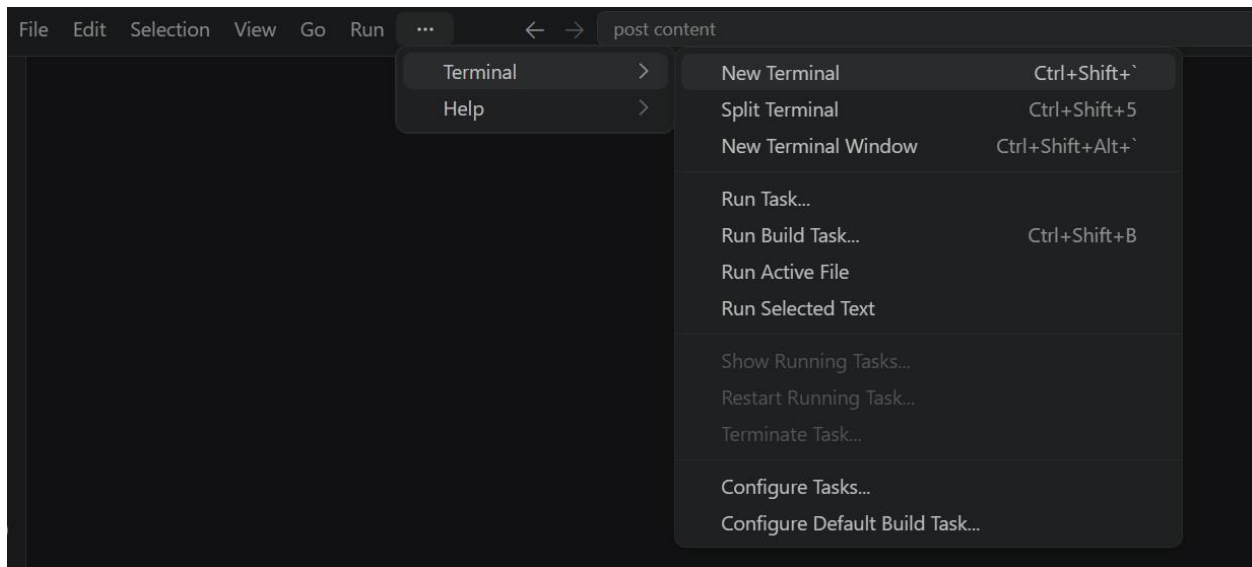
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Rising Waters: A Machine Learning Approach to Flood Prediction application locally to verify its functionality. This involves setting up the project environment, installing the required dependencies, running the Flask application, and testing the flood prediction system through a web browser.

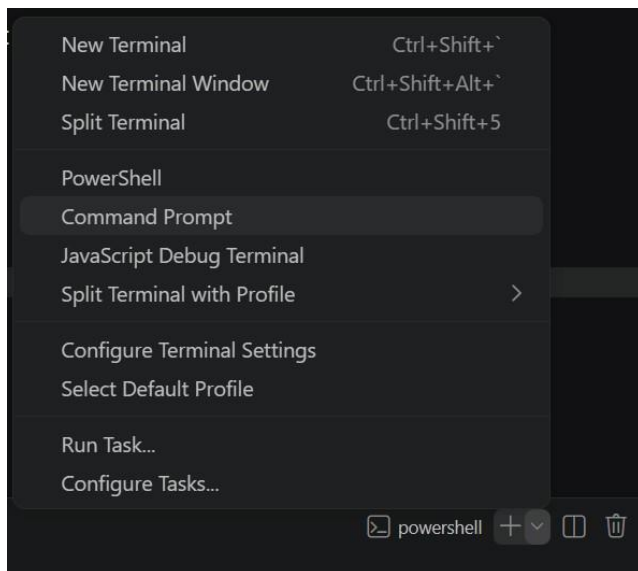
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



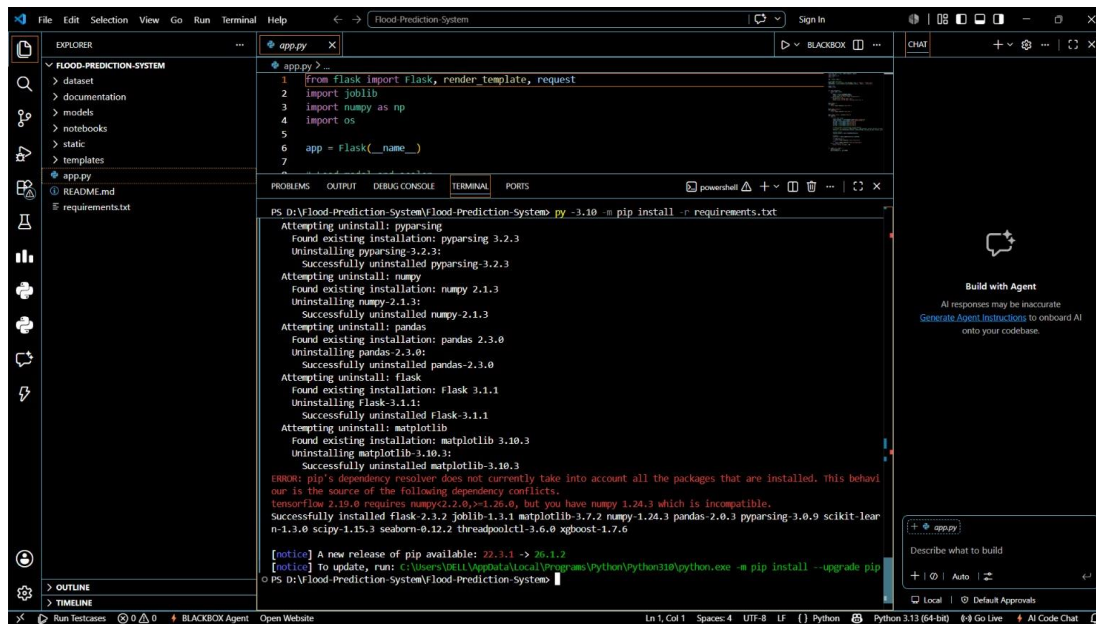
- Then open “Command Prompt”



- Navigate to the project directory.
- Install all the required dependencies using the following command:

```
pip install -r requirements.txt
pip install -r requirements.txt
```

- This installs all the required Python libraries such as Flask, NumPy, Pandas, Scikit-learn, Joblib, and other project dependencies



```

1 from flask import Flask, render_template, request
2 import joblib
3 import numpy as np
4 import os
5
6 app = Flask(__name__)
7
PS D:\Vlood-Prediction-System\Flood-Prediction-System> py -3.10 -m pip install -r requirements.txt
Attempting uninstall: pyparsing
Found existing installation: pyparsing 3.2.3
Uninstalling pyparsing-3.2.3:
Successfully uninstalled pyparsing-3.2.3
Attempting uninstall: numpy
Found existing installation: numpy 2.1.3
Uninstalling numpy-2.1.3:
Successfully uninstalled numpy-2.1.3
Attempting uninstall: pandas
Found existing installation: pandas 2.3.0
Uninstalling pandas-2.3.0:
Successfully uninstalled pandas-2.3.0
Attempting uninstall: flask
Found existing installation: flask 3.1.1
Uninstalling flask-3.1.1:
Successfully uninstalled flask-3.1.1
Attempting uninstall: matplotlib
Found existing installation: matplotlib 3.10.3
Uninstalling matplotlib-3.10.3:
Successfully uninstalled matplotlib-3.10.3
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed, this behavior
is the source of the following dependency conflicts.
tensorflow 2.15.0 requires numpy>2.0.0, but you have numpy 1.24.3 which is incompatible.
Successfully installed flask-2.3.2 joblib-1.3.1 matplotlib-3.7.2 numpy-1.24.3 pandas-2.0.3 pyparsing-3.0.9 scikit-learn-
n-1.3.0 scipy-1.15.3 seaborn-0.12.2 threadpoolctl-3.6.0 xgboost-1.7.6

[notice] A new release of pip available: 22.3.1 -> 26.1.2
[notice] To update, run: C:\Users\BULL\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip
PS D:\Vlood-Prediction-System\Flood-Prediction-System>
  
```

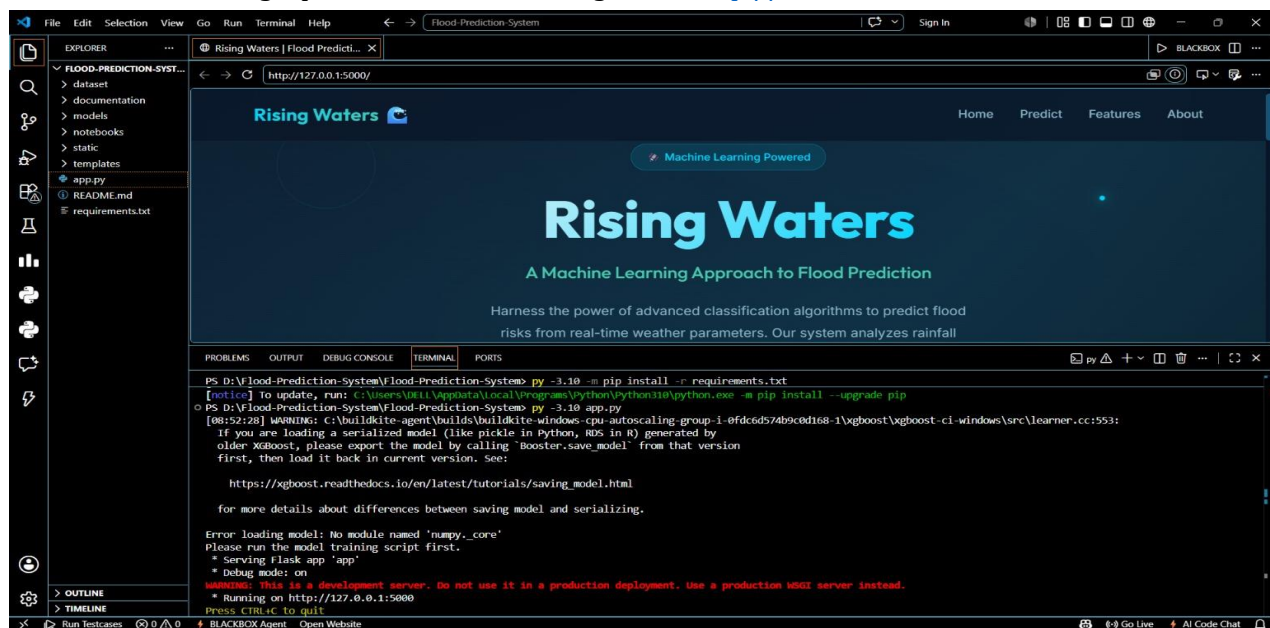
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
python app.py
```

- Once running, open a browser and navigate to ["http://127.0.0.1:5050"](http://127.0.0.1:5050) to access the a



```

PS D:\Vlood-Prediction-System\Flood-Prediction-System> py -3.10 -m pip install -r requirements.txt
[notice] To update, run: C:\Users\BULL\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip
PS D:\Vlood-Prediction-System\Flood-Prediction-System> py -3.10 app.py
[08:52:28] WARNING: C:\buildkite-agent\builds\buildkite-windows-cpi-autoscaling-group-1-0fd6c6574b9cd168-1\wxboost\wxboost-ci-windows\src\learner.cc:553:
If you are loading a serialized model (like pickle in Python, RDS in R) generated by
older XGBoost, please export the model by calling 'Booster.save_model' from that version
first, then load it back in current version. See:
https://xgboost.readthedocs.io/en/latest/tutorials/saving_model.html
for more details about differences between saving model and serializing.
Error loading model: No module named 'numpy.core'
Please run the model training script first.
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5050
Press CTRL+C to quit
  
```


- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Cloud Deployment Using Render

The **Rising Waters: A Machine Learning Approach to Flood Prediction** application was successfully deployed on the **Render** cloud platform to provide public access over the internet. Cloud deployment allows users to access the application without installing Python or project dependencies on their local systems.

Deployment Steps

- Push the project source code to the GitHub repository.
- Create a new **Web Service** in Render.
- Connect the GitHub repository to Render.
- Set the **Root Directory** to:

Rising_Waters_Project_Files

- Configure the Build Command: **pip install -r requirements.txt**
- Configure the Start Command: **gunicorn app:app**
- Deploy the application and verify successful deployment.

Deployment Verification

- Open the deployed application using the Render URL.
- Navigate to the prediction page.
- Enter the required weather parameters.
- Click **Predict**.
- Verify that the application correctly displays either **Flood Risk** or **No Flood Risk** based on the provided inputs.

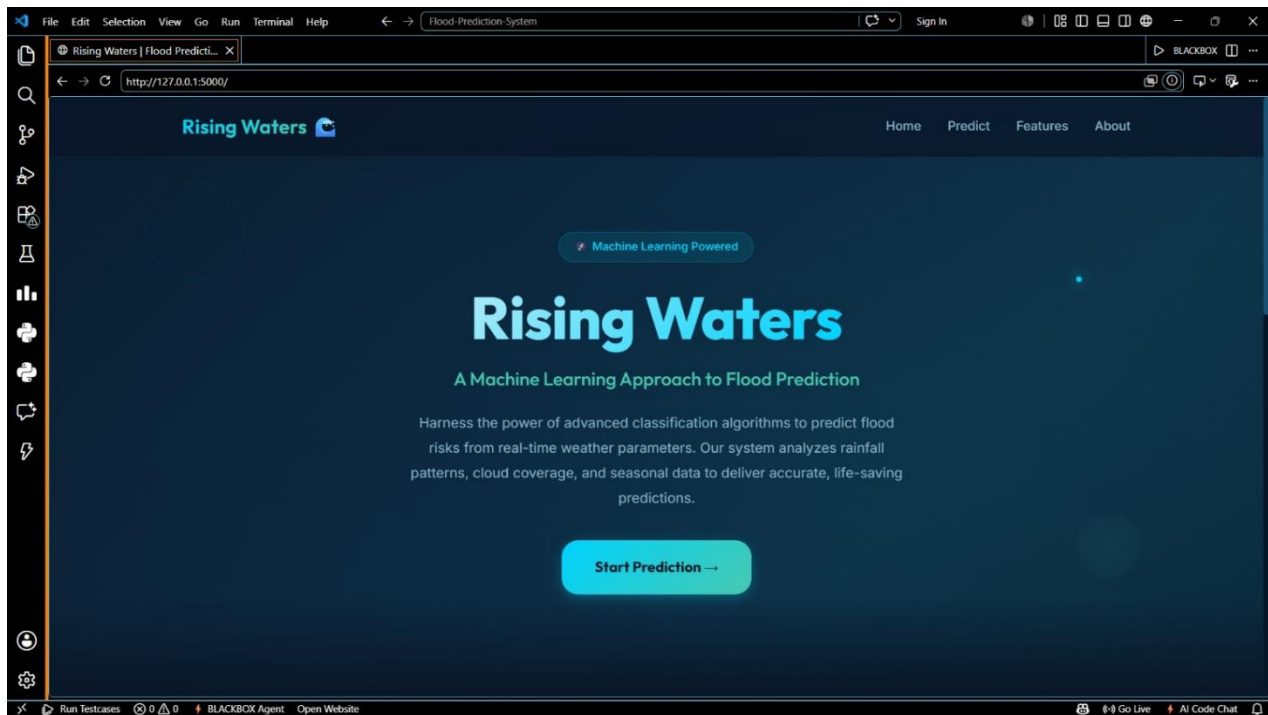
Deployment URL: <https://rising-waters-hh79.onrender.com/>

Outcome

The application was successfully deployed on the Render cloud platform and is accessible through web browser. The deployed system performs flood prediction using the trained XGBoost machine learning model and provides prediction results through an interactive Flask web interface.

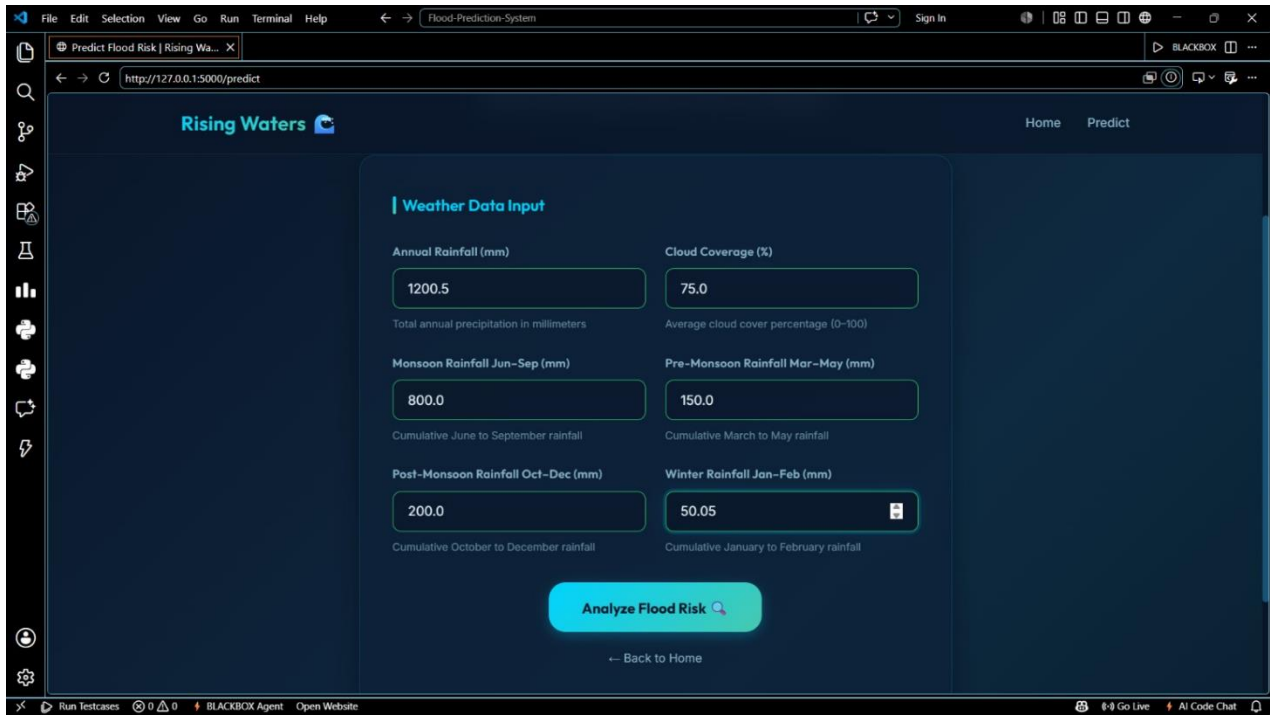
Exploring the Website's Web Pages:

Home / Generator Page:



Description: The Home Page serves as the landing page of the **Rising Waters: A Machine Learning Approach to Flood Prediction** application. It provides a clean and user-friendly interface introducing the purpose of the system and its role in predicting flood risk using Machine Learning. The page includes the project title, a brief description, and a navigation button that directs users to the flood prediction page.

Input Section:



The screenshot shows a web browser window displaying the 'Rising Waters' application. The page is titled 'Weather Data Input' and contains several input fields for meteorological data. The values entered in the fields are: Annual Rainfall (1200.5), Cloud Coverage (75.0), Monsoon Rainfall (800.0), Pre-Monsoon Rainfall (150.0), Post-Monsoon Rainfall (200.0), and Winter Rainfall (50.05). A large green button labeled 'Analyze Flood Risk' is positioned below the input fields. The browser's address bar shows the URL 'http://127.0.0.1:5000/predict'. The application's navigation bar includes 'Home' and 'Predict' links. The footer of the browser window shows 'Run Testcases', 'BLACKBOX Agent', and 'Open Website' buttons.

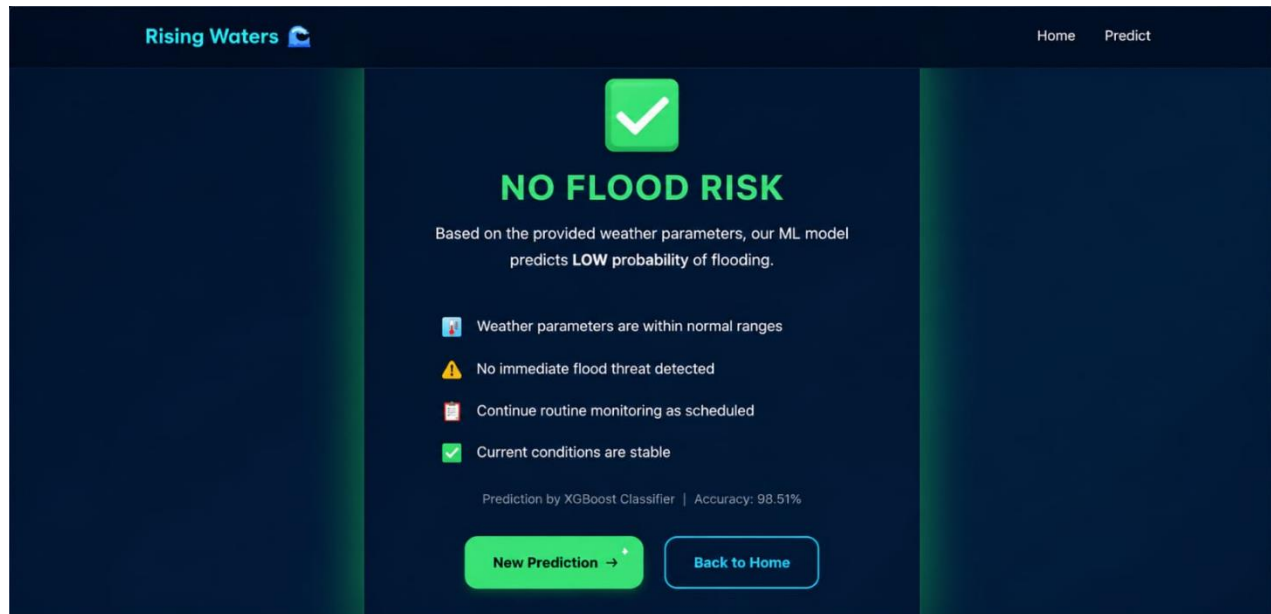
Weather Data Input	
Annual Rainfall (mm) Total annual precipitation in millimeters	Cloud Coverage (%) Average cloud cover percentage (0-100)
Monsoon Rainfall Jun-Sep (mm) Cumulative June to September rainfall	Pre-Monsoon Rainfall Mar-May (mm) Cumulative March to May rainfall
Post-Monsoon Rainfall Oct-Dec (mm) Cumulative October to December rainfall	Winter Rainfall Jan-Feb (mm) Cumulative January to February rainfall

Analyze Flood Risk

[Back to Home](#)

Description: The Input Page contains a user-friendly form where users enter the required meteorological parameters for flood prediction. The form includes input fields for **Annual Rainfall**, **Cloud Coverage**, **June–September Rainfall**, **March–May Rainfall**, **October–December Rainfall**, and **January–February Rainfall**. After entering the required values, users click the **Predict** button to submit the data to the Flask backend, where it is processed by the trained **XGBoost** machine learning model.

Results Section:



Description: The Results Page displays the flood prediction generated by the trained Machine Learning model. Based on the processed rainfall and weather data, the application presents either a **Flood Chance** or **No Flood Chance** result. Separate result pages provide a clear and user-friendly indication of the predicted flood risk, enabling users to understand the outcome quickly and effectively.

Conclusion:

The **Rising Waters: A Machine Learning Approach to Flood Prediction** system is a Flask-based web application that uses the **XGBoost Machine Learning algorithm** to predict flood risk from rainfall and meteorological data. The application combines data preprocessing, model training, and real-time prediction to provide accurate flood risk assessments through an intuitive web interface. By enabling early flood prediction, the system supports disaster preparedness and informed decision-making. Future enhancements may include real-time weather API integration, cloud deployment, GIS-based flood visualization, mobile application support, and automated early warning notifications to improve accessibility and prediction capabilities.